# Final Project - Data prog with R

Charalampos Sarakatsanis

## Data Prog with R 2025/2026

STAT30340/STAT40730 Data Programming with R

Link for the dataset of Part1: https://data.gov.ie/dataset/traffic-volumes-from-scats-traffic-management-system-2022

Link for the dataset of part 2: https://www.kaggle.com/datasets/hibrahimag1/top-1000-twitch-streamers-data-may-2024

## Part 1

The dataset I chose from data.gov.ie for Part 1 is called traffic-volumes-from-scats-traffic-management-system-2022 and I chose the data from November 2022. This dataset contains traffic volume data collected by the Sydney Coordinated Adaptive Traffic System (SCATS) traffic detectors located at junctions for the area "Dun Laoghaire–Rathdown" for the month November of 2022. The dataset contains both categorical and numerical variables.

- The categorical variables are: **region** (the geographical area where the traffic measurement happened), **site** (identifies the junction or the location where the traffic detector is installed), and **detector** (identifies the specific detector), which describe the location of each traffic sensor.
- The numerical variables in the dataset are: **hour** (a numerical time-based variable derived from the timestamp **End_Time** (it displays the hour of the day from 0 to 23), **Sum_Volume** (the total number of vehicles recorded for each measurement interval), and **Avg_Volume** (the average traffic volume).

```
#Importing the libraries that are going to be used.
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```
library(dplyr)
library(lubridate)
library(skimr)
library(purrr)
library(ggplot2)
```

```
#importing the dataset.
df <- as_tibble(read.csv("scats_dlr_nov_2022.csv"))#I only displayed the End_Time value, bec
#we will see that there will be null values due to mistakes during the creating of the datas
unique(is.na(df$End_Time))
```

```
[1] FALSE
```

We will change the dataset to a more suitable form in order to analyse and visualise it.

```
df$End_Time = ymd_hms(df$End_Time)
df$Hour = hour(df$End_Time)
df$Wday = wday(df$End_Time, label = TRUE)
df$Wday = factor(df$Wday, ordered= FALSE)
df$Site = factor(df$Site)
df$Detector = factor(df$Detector)
df$Region = factor(df$Region)
```

```
skim(df)
```

Table 1: Data summary

| | |
|---|---|
| Name | df |
| Number of rows | 2263414 |

| | |
|---|---|
| Number of columns | 8 |
| Column type frequency: | |
| factor | 4 |
| numeric | 3 |
| POSIXct | 1 |
| Group variables | None |

**Variable type: factor**

| skim_variable | n_missing | complete_rate | ordered | n_unique | top_counts |
|---|---|---|---|---|---|
| Region | 0 | 1 | FALSE | 3 | IRE: 2156271, IRE: 90119, SCI: 17024 |
| Site | 0 | 1 | FALSE | 220 | 621: 23040, 636: 23040, 636: 23040, 636: 23040 |
| Detector | 0 | 1 | FALSE | 32 | 2: 149873, 3: 149535, 1: 149288, 4: 146827 |
| Wday | 0 | 1 | FALSE | 7 | Tue: 376279, Wed: 375631, Sun: 302829, Mon: 302814 |

**Variable type: numeric**

| skim_variable | n_missing | complete_rate | mean | sd | p0 | p25 | p50 | p75 | p100 | hist |
|---|---|---|---|---|---|---|---|---|---|---|
| Sum_Volume | 0 | 1 | 54.88 | 126.23 | 0 | 0 | 1 | 42 | 3415 | |
| Avg_Volume | 0 | 1 | 4.40 | 10.53 | 0 | 0 | 0 | 3 | 379 | |
| Hour | 0 | 1 | 11.51 | 6.92 | 0 | 6 | 12 | 18 | 23 | |

**Variable type: POSIXct**

| skim_variable | n_missing | complete_rate | min | max | median | n_unique |
|---|---|---|---|---|---|---|
| End_Time | 0 | 1 | 2022-11-01 | 2022-11-30 23:00:00 | 2022-11-16 02:00:00 | 720 |

The issue comes from the way the variable **Wday** was created. Even though **End_Time** does not contain any NA values, **Wday** does, due to a small mistake during the preprocessing stage when the dataset was first created. There were attempts to fix this later, but the variable was already affected.

The percentage of missing data is small, and because the problem was caused by the same variable, all variables that were derived from it have the exact same number of missing values. As a result, the same rows are missing for each individual variable. The safest and most straightforward way to fix this, is to remove those rows.

```
df = na.omit(df)
```

Now that we have no missing values, we can create linear models to extract information about linear relations between the data using summary.

```
#creating the linear models
sv_model = lm(Sum_Volume ~ Avg_Volume+Wday+Hour+Region, data = df)
av_model = lm(Avg_Volume ~ Sum_Volume+Wday+Hour+Region, data = df)
#summarising the linear models
summary(sv_model)
```

```
Call:
lm(formula = Sum_Volume ~ Avg_Volume + Wday + Hour + Region,
    data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-2517.04    -2.47    -1.34     2.87    24.67

Coefficients:
              Estimate Std. Error   t value Pr(>|t|)
(Intercept)  2.2755096  0.0244233    93.169  < 2e-16 ***
Avg_Volume  11.9393430  0.0007333 16282.123  < 2e-16 ***
WdayMon     -0.1563903  0.0294342    -5.313 1.08e-07 ***
WdayTue      0.0063479  0.0279640     0.227  0.82042
WdayWed     -0.1021846  0.0279762    -3.653  0.00026 ***
WdayThu     -0.1281308  0.0294568    -4.350 1.36e-05 ***
WdayFri      0.1856796  0.0294445     6.306 2.86e-10 ***
WdaySat     -1.3088522  0.0294715   -44.411  < 2e-16 ***
Hour         0.0235861  0.0011144    21.165  < 2e-16 ***
RegionIRE3   0.0475935  0.0389584     1.222  0.22184
```

```
RegionSCITY  1.3385406  0.0881157    15.191  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 11.45 on 2263403 degrees of freedom
Multiple R-squared:  0.9918,    Adjusted R-squared:  0.9918
F-statistic: 2.728e+07 on 10 and 2263403 DF,  p-value: < 2.2e-16
```

```r
summary(av_model)
```

```
Call:
lm(formula = Avg_Volume ~ Sum_Volume + Wday + Hour + Region,
    data = df)

Residuals:
    Min      1Q  Median      3Q     Max
 -0.858  -0.227   0.093   0.171 211.140

Coefficients:
              Estimate Std. Error   t value Pr(>|t|)
(Intercept) -1.823e-01  2.037e-03   -89.498  < 2e-16 ***
Sum_Volume   8.305e-02  5.101e-06 16282.123  < 2e-16 ***
WdayMon      1.913e-02  2.455e-03     7.794 6.50e-15 ***
WdayTue      7.155e-03  2.332e-03     3.068 0.002155 **
WdayWed      1.685e-02  2.333e-03     7.221 5.16e-13 ***
WdayThu      1.903e-02  2.457e-03     7.745 9.56e-15 ***
WdayFri     -6.619e-03  2.456e-03    -2.695 0.007034 **
WdaySat      1.134e-01  2.458e-03    46.157  < 2e-16 ***
Hour         9.432e-05  9.295e-05     1.015 0.310216
RegionIRE3   1.147e-02  3.249e-03     3.531 0.000414 ***
RegionSCITY -1.202e-01  7.349e-03   -16.351  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9551 on 2263403 degrees of freedom
Multiple R-squared:  0.9918,    Adjusted R-squared:  0.9918
F-statistic: 2.727e+07 on 10 and 2263403 DF,  p-value: < 2.2e-16
```

The p-values indicate which variables show strong statistical evidence of association with traffic volume (sum or average), while the estimates describe direction and the relative magnitude

of these associations. By summarising a simple linear model, we can detect pure and linear relations that will help determine the most interesting choices for plotting later.

```
num_cols <- c("Sum_Volume", "Avg_Volume", "Hour")

# We will use purrr::map and map_dbl in order to create a correlation table
cor_table <- map(
  num_cols,
  function(var1) {
    map_dbl(
      num_cols,
      function(var2) {
        cor(df[[var1]], df[[var2]], use = "complete.obs")
      }
    )
  }
)

cor_table
```

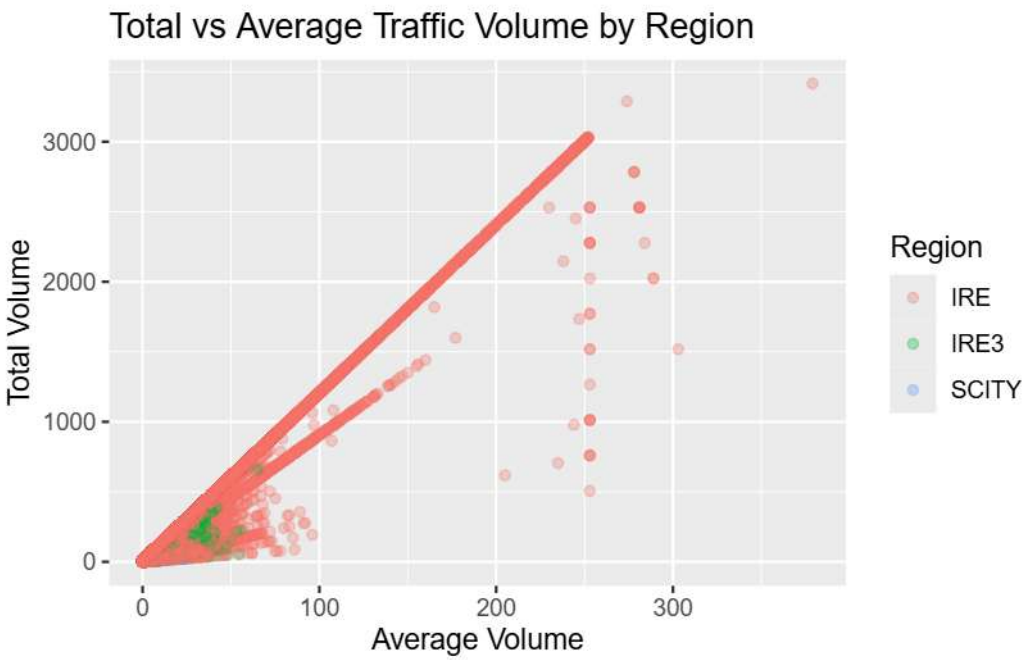
```
[[1]]
[1] 1.0000000 0.9958694 0.1600143

[[2]]
[1] 0.9958694 1.0000000 0.1594157

[[3]]
[1] 0.1600143 0.1594157 1.0000000
```

From this correlation table, we can understand from the relation between **`Sum_Volume`** and **`Avg_Volume`** that there is no noise or that the amount of noise is insignificant. From the fact that **`hour`** does not correlate with the volume, we should assume that there is no correlation, but the p-value reveals that we gain information out of it. This means that there are a few hours in the day that correlate highly with the dataset and the others are insignificant. That would be a great case to visualise.

Another interesting case to visualise is **`weekdays`**, that similarly with `hour`, a few days correlate with the volume and after that the other days are insignificant, so we can gain information out of visualising that relation also.

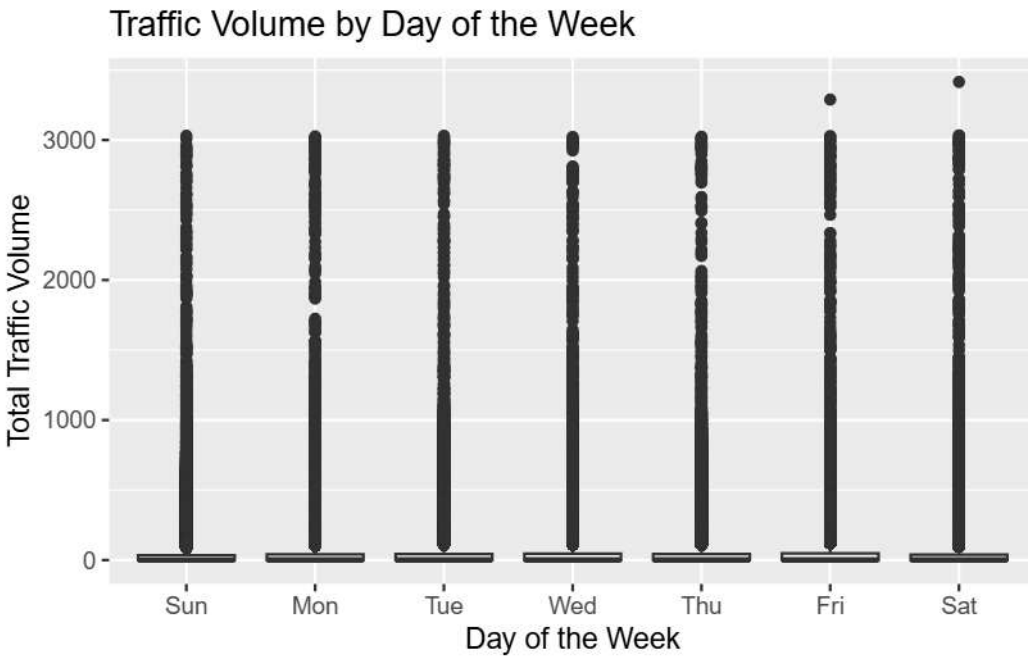
Lastly, the same is happening with **`Region`**. Using a box plot, since we only have 3 regions, will provide us with a plethora of information.

```r
ggplot(df, aes(x = Avg_Volume, y = Sum_Volume, color = Region)) +
  geom_point(alpha = 0.3) +
  labs(
    title = "Total vs Average Traffic Volume by Region",
    x = "Average Volume",
    y = "Total Volume"
  )
```



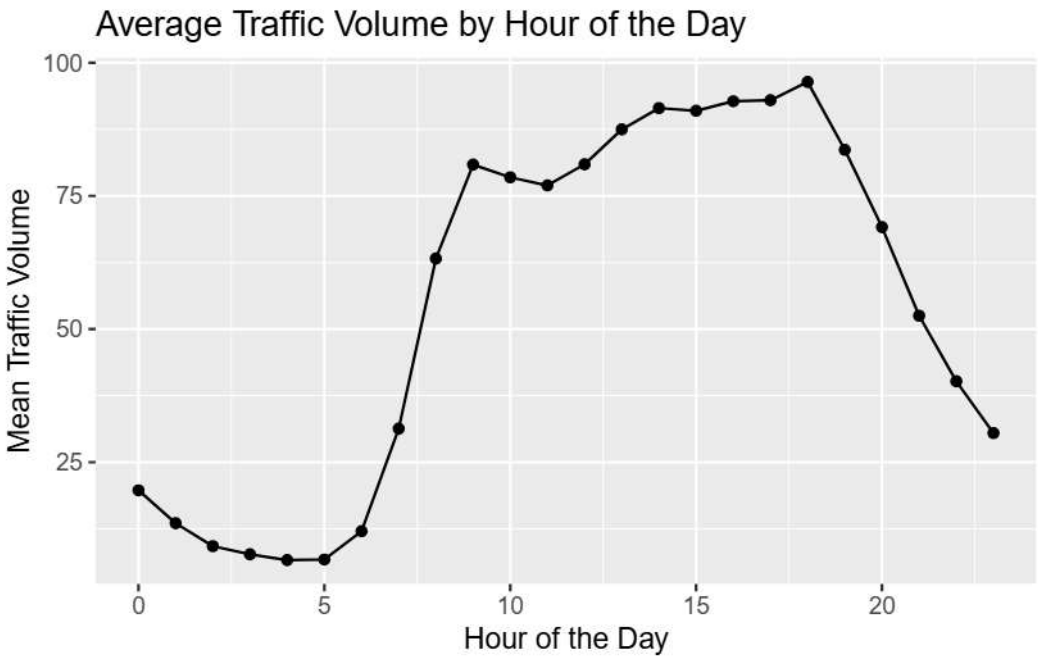

From this plot, we can see that there are **three different linear relationships** between the average traffic volume and the total. Also, almost all of the traffic is detected in the **IRE** region. **IRE3** follows with less than a quarter of the observations, while **SCITY** is last and almost undetectable in the plot. This suggests that regional differences may explain part of the structure observed in the overall relationship between average and total traffic volume.

```r
ggplot(df, aes(x = Wday, y = Sum_Volume)) +
  geom_boxplot() +
  labs(
    title = "Traffic Volume by Day of the Week",
    x = "Day of the Week",
    y = "Total Traffic Volume"
  )
```




Even though there there were differences in the significance of each weekday, we can see that overall each day has very similar traffic

```
df |>
  group_by(Hour) |>
  summarise(mean_volume = mean(Sum_Volume, na.rm = TRUE)) |>
  ggplot(aes(x = Hour, y = mean_volume)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Average Traffic Volume by Hour of the Day",
    x = "Hour of the Day",
    y = "Mean Traffic Volume"
  )
```




The case of **`hour`** is different from **`weekdays`**. Visually, it is very clear that there are peak traffic hours and very low traffic as well. From **06:00 to 09:00** there is a rapid increase in traffic, and from **07:00** the traffic starts to become more noticeable. There is a small negative curve from **09:00 to 11:00**, but the traffic volume is still kept at a high level, and after that there is only a positive increase rate until **18:00**, where the peak traffic hour is reached. Afterwards, an accelerated decrease follows until **05:00**, with the traffic being kept at a low level from **01:00**.

## Part 2 & 3

Our dataset describes data from the 1000 most popular streamers of 2023, in ascending order of popularity. To be more precise, the information we are provided with is:

Streamer name, language, type of stream the streamer mainly does, most streamed game/activity, second most streamed game/activity, average stream duration, average number of followers gained from each stream, average viewers per stream, average games/activities per stream, total time streamed, total followers, total views, total games/activities streamed, active days per week, most active day, and the day with the most followers gained.

```
#importing the dataset
df = read.csv("datasetV2.csv")
```

```r
#checking if there are any null values for each variable.
unique(is.na(df))
```

```
      RANK  NAME LANGUAGE  TYPE MOST_STREAMED_GAME X2ND_MOST_STREAMED_GAME
[1,] FALSE FALSE    FALSE FALSE              FALSE                   FALSE
     AVERAGE_STREAM_DURATION FOLLOWERS_GAINED_PER_STREAM AVG_VIEWERS_PER_STREAM
[1,]                   FALSE                       FALSE                  FALSE
     AVG_GAMES_PER_STREAM TOTAL_TIME_STREAMED TOTAL_FOLLOWERS TOTAL_VIEWS
[1,]                FALSE               FALSE           FALSE       FALSE
     TOTAL_GAMES_STREAMED ACTIVE_DAYS_PER_WEEK MOST_ACTIVE_DAY
[1,]                FALSE                FALSE           FALSE
     DAY_WITH_MOST_FOLLOWERS_GAINED
[1,]                          FALSE
```

There are no null values.

Now lets give a more convenient from to our dataset, by turning into factors all of the categorical variables, except from the `NAME`.

```r
df$LANGUAGE = factor(df$LANGUAGE)
df$TYPE = factor(df$TYPE)
df$MOST_STREAMED_GAME = factor(df$MOST_STREAMED_GAME)
df$X2ND_MOST_STREAMED_GAME = factor(df$X2ND_MOST_STREAMED_GAME)
```

Lets check the number of categorical values of each factor variable.

```r
length(unique(df$LANGUAGE))
```

```
[1] 19
```

```r
length(unique(df$TYPE))
```

```
[1] 2
```

```r
length(unique(df$MOST_STREAMED_GAME))
```

```
[1] 105
```

```r
length(unique(df$X2ND_MOST_STREAMED_GAME))
```

```
[1] 179
```

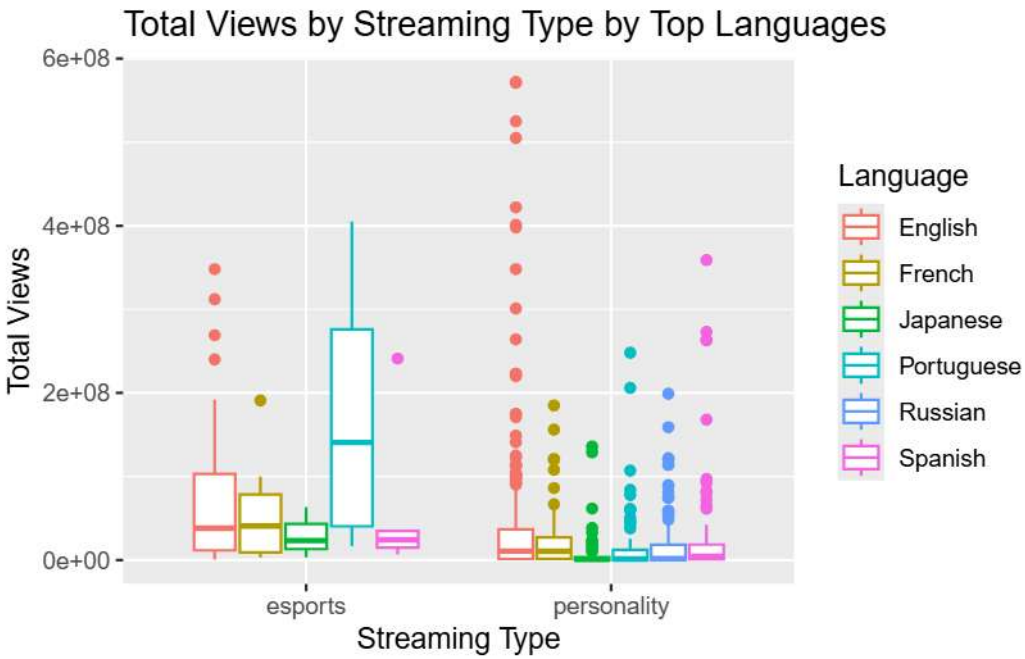
Judging from the outcome, an interesting approach would be is to visualise the total streamer's views per streaming type. In order to view the popularity of each streaming type. This is a very convenient variable for simple plotting and prediction, since the outcome can be personality or esports.

The same can be done with language.

```r
library(dplyr)
library(ggplot2)
library(scales)
```

```
Attaching package: 'scales'

The following object is masked from 'package:purrr':

    discard

The following object is masked from 'package:readr':

    col_factor
```

```r
plotdf <- df |>
  count(LANGUAGE, sort = TRUE) |>
  slice_head(n = 6) |>
  pull(LANGUAGE)

df |>
  filter(LANGUAGE %in% plotdf) |>
  ggplot(aes(x = TYPE, y = TOTAL_VIEWS, color = LANGUAGE)) +
  geom_boxplot() +
  labs(
    title = "Total Views by Streaming Type by Top Languages",
    x = "Streaming Type",
    y = "Total Views",
    color = "Language"
  )
```

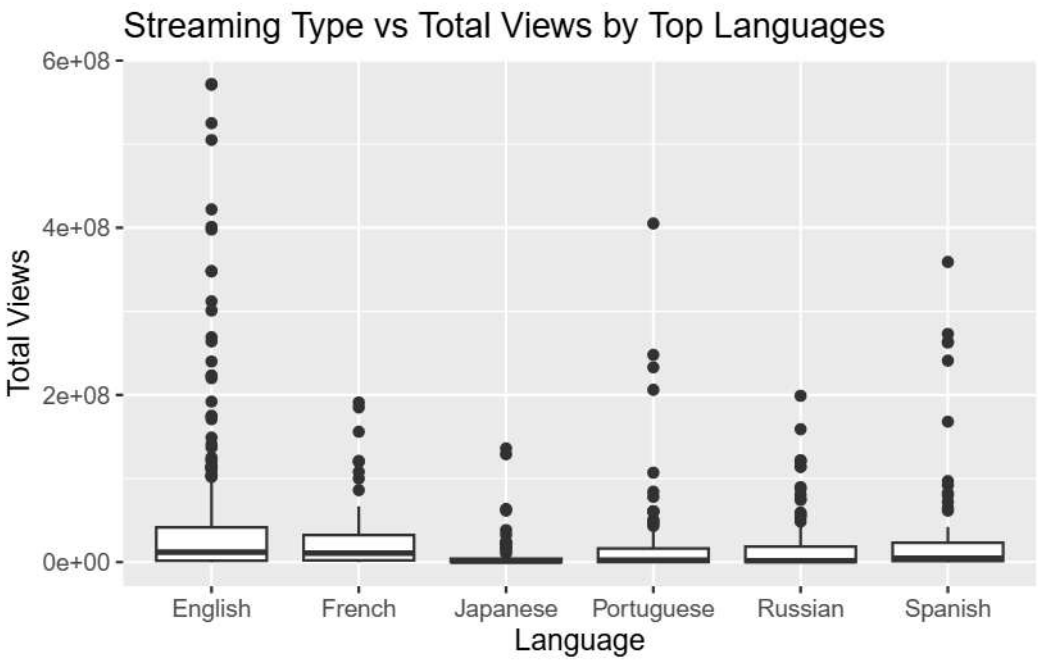



From this graph, we can see that there are no streams in Russian that are related to esports. Apart from that, streamers that focus on esports tend to receive from roughly twice up to more than ten times the number of views, depending on the language. Due to the diversity of streamers and the variation within each language, the standard deviation in all cases is close to the mean. This indicates that as the mean number of views increases, the standard deviation also increases.

On the other hand, the most successful cases are observed in English, driven by a small number of individuals who manage to accumulate extremely large numbers of views through personality-type streams. Similar but much less dense patterns can also be observed in other languages. This suggests that esports streams generally have a higher and more consistent chance of success, while personality streams carry a smaller probability of extremely high success, but with potentially much larger peaks.

```
plotdf <- df |>
  count(LANGUAGE, sort = TRUE) |>
  slice_head(n = 6) |>
  pull(LANGUAGE)

df |>
  filter(LANGUAGE %in% plotdf) |>
  ggplot(aes(x = LANGUAGE, y = TOTAL_VIEWS)) +
  geom_boxplot() +
  labs(
```

```
    title = "Streaming Type vs Total Views by Top Languages",
    x = "Language",
    y = "Total Views"
  )
```




### Caret

The caret package (Classification And REgression Training) contains functions for complex regression and classification problems. The package "suggests" field includes 31 packages.

We will install the suggested package by using the following code chunk, as its vignette recommends:

`{install.packages("caret", dependencies = c("Depends", "Suggests"))}`

Two of the most basic and commonly used tools of caret are `train` and `createDataPartition`.

- `createDataPartition`, as the name suggests, helps us split the data into different sets for training and testing using a seed.
- `train` is used for evaluating and choosing the optimal model across different parameters (`train` works by using resampling to study the effect of model tuning parameters on performance), and it estimates model performance from a training set. More information

about this algorithm can be found in the caret manual: https://topepo.github.io/caret/model-training-and-tuning.html#model-training-and-parameter-tuning

There are many different ways when using `train` to choose the method (type of model) and the type of resampling, such as 10-fold cross-validation, and to specify the methods for performance measurement. In order to do that, the three most convenient and popular choices are `trainControl`, `defaultSummary`, and `twoClassSummary`.

The package has several functions for visualising, with one common method being a ggplot function for train objects.

In order to predict new data samples, we use `predict.train`. There are functions to compute different estimates, the confusion matrix, and generally many different model statistics.

### Reasons to use caret

- The produced code is easy to understand, and its structure is suitable for reporting.
- It is very easy to change the training code in order to create different models, add or remove evaluation methods, and apply different types of resampling.
- There is a plethora of different training and evaluation methods that can be chosen, from basic models to more advanced algorithms and configurations.

### Using caret

Based on the dataset with the 1000 most popular streamers, we can use caret in order to create two models. One that predicts average stream duration and one that predicts the stream type

For the stream type, since will will mostly use categorical data in order to find it, so knn we would be the natural choice, but since we are handling important categorical variables, we will go with random forest. This way clustering will be really helpful to create an accurate model. Also there are two different stream types: `personality` and `esports`, which is also a very convenient number of labels to predict for a 1000 entry dataset.

Now let's create the train an test sets for the model that predicts the stream type.

```
library(caret)
```

```
Loading required package: lattice


Attaching package: 'caret'
```

```
The following object is masked from 'package:purrr':

    lift
```

```r
library(mlbench)
#setting the seed
set.seed(123)

#creating the train set
inTrain = createDataPartition(
  #data that will be predicted
  y = df$TYPE,
  #the percentage of data that will be used to train the model
  #We used 0.8 which is a little high, because we have 1000 different entries.
  p = 0.8,
  #The outcome we will produce will not be a list, but `personality` or `esports`
  list = FALSE
)
#We remove the NAME, because there are a lot of different names #that the model will take ac
#categories and will become a lot slower. Also each streamer has a different name, so it wil
training = df[inTrain,] |> select(-NAME)
testing = df[-inTrain,] |> select(-NAME)
```

Since we do not have enormous amounts of data, we will train a model with all the variables of the dataset and based on evaluations and estimators, we will create the final model.

```r
rfmodel = train(
  #choosing all the variables as predictors and TYPE(stream type)
  #as the variable that will be predicted.
  TYPE ~.,
  data = training,
  method = "rf",
  ntree=50
)
```

```r
pred = predict(rfmodel, newdata = testing)
confusionMatrix(pred, testing$TYPE)
```

```
Confusion Matrix and Statistics

              Reference
```

```
Prediction    esports personality
  esports           8           0
  personality       3         188

               Accuracy : 0.9849
                 95% CI : (0.9566, 0.9969)
    No Information Rate : 0.9447
    P-Value [Acc > NIR] : 0.004132

                  Kappa : 0.8344

 Mcnemar's Test P-Value : 0.248213

            Sensitivity : 0.72727
            Specificity : 1.00000
         Pos Pred Value : 1.00000
         Neg Pred Value : 0.98429
             Prevalence : 0.05528
         Detection Rate : 0.04020
   Detection Prevalence : 0.04020
      Balanced Accuracy : 0.86364

       'Positive' Class : esports
```

```r
varImp(rfmodel)
```

```
rf variable importance

  only 20 most important variables shown (out of 322)

                                                Overall
AVG_VIEWERS_PER_STREAM                          100.000
TOTAL_GAMES_STREAMED                             44.763
TOTAL_VIEWS                                      27.826
AVG_GAMES_PER_STREAM                             21.575
TOTAL_FOLLOWERS                                  16.287
RANK                                             12.299
AVERAGE_STREAM_DURATION                          10.711
TOTAL_TIME_STREAMED                               8.784
FOLLOWERS_GAINED_PER_STREAM                       7.728
ACTIVE_DAYS_PER_WEEK                              6.903
```

```
X2ND_MOST_STREAMED_GAMESea of Thieves            5.901
X2ND_MOST_STREAMED_GAMESuper Smash Bros. Melee   5.415
X2ND_MOST_STREAMED_GAMERocket League Sideswipe   4.759
MOST_STREAMED_GAMEValorant                       4.078
X2ND_MOST_STREAMED_GAMEClone Hero                4.060
X2ND_MOST_STREAMED_GAMEVariety                   3.817
LANGUAGEFrench                                   2.923
X2ND_MOST_STREAMED_GAMELegends of Runeterra      2.485
MOST_STREAMED_GAMEBrawlhalla                     2.408
MOST_STREAMED_GAMESuper Smash Bros. Ultimate     2.282
```

Based on the variable importance overall scores from varImp, all features provide us with meaningful information, and none showed us negligible importance values. Since the Random Forest model is lightweight due to the limited number of trees, no variables need to be removed.

Since we have already plenty of categorical variables that can be useful, and numerical in order to find the average steam duration.

Using the same split as before is a wise and time-saving move, but this time we will change y (the variable we want to predict).

```r
library(caret)
library(mlbench)
#setting the seed
set.seed(123)

#creating the train set
inTrain = createDataPartition(
  #data that will be predicted
  y = df$AVERAGE_STREAM_DURATION,
  #the percentage of data that will be used to train the model
  #We used 0.8 which is a little high, because we have 1000 different entries.
  p = 0.8,
  #The outcome we will produce will not be a list, but `personality` or `esports`
  list = FALSE
)
#We remove the NAME, because there are a lot of different cames #that the model will take ac
#different categories and will become a lot slower. Also each streamer has a different name,
#give us new insight.
training = df[inTrain,] |> select(-NAME)
testing = df[-inTrain,] |> select(-NAME)
```

```r
rfmodel2 = train(
  #choosing all the variables as predictors and TYPE(stream type)
  #as the variable that will be predicted.
  AVERAGE_STREAM_DURATION ~.,
  data = training,
  method = "rf",
  ntree=300
)
```

```r
pred = predict(rfmodel2, newdata = testing)
postResample(pred, testing$AVERAGE_STREAM_DURATION)
```

```
     RMSE  Rsquared       MAE
2.1456869 0.5573636 1.3193569
```

```r
varImp(rfmodel2)
```

```
rf variable importance

  only 20 most important variables shown (out of 322)

                                                Overall
TOTAL_TIME_STREAMED                             100.000
MOST_STREAMED_GAMEVirtual Casino                 52.722
ACTIVE_DAYS_PER_WEEK                             49.566
TOTAL_FOLLOWERS                                  36.970
TOTAL_GAMES_STREAMED                             21.356
AVG_GAMES_PER_STREAM                             20.643
AVG_VIEWERS_PER_STREAM                           18.511
LANGUAGEPortuguese                               17.044
FOLLOWERS_GAINED_PER_STREAM                      13.485
TOTAL_VIEWS                                      13.192
RANK                                              9.243
MOST_STREAMED_GAMEDota 2                          5.279
X2ND_MOST_STREAMED_GAMEFINAL FANTASY XIV ONLINE   4.972
LANGUAGEEnglish                                   4.572
MOST_STREAMED_GAMEAnimals Aquariums and Zoos      4.279
X2ND_MOST_STREAMED_GAMEAlbion Online              3.948
MOST_STREAMED_GAMEVariety                         3.751
TYPEpersonality                                   3.738
MOST_STREAMED_GAMESlots                           2.039
```

```
MOST_ACTIVE_DAYThursday                          2.009
```

```r
rfmodelavgviews = train(
  #choosing all the variables as predictors and TYPE(stream type)
  #as the variable that will be predicted.
  AVERAGE_STREAM_DURATION ~.,
  data = training,
  method = "rf",
  ntree=100
)
```

```r
pred = predict(rfmodelavgviews, newdata = testing)
postResample(pred, testing$AVERAGE_STREAM_DURATION)
```

```
     RMSE  Rsquared       MAE
2.1332078 0.5634994 1.3294601
```

```r
varImp(rfmodelavgviews)
```

```
rf variable importance

  only 20 most important variables shown (out of 322)

                                                Overall
TOTAL_TIME_STREAMED                             100.000
MOST_STREAMED_GAMEVirtual Casino                 53.832
ACTIVE_DAYS_PER_WEEK                             41.795
TOTAL_FOLLOWERS                                  33.915
AVG_GAMES_PER_STREAM                             19.500
AVG_VIEWERS_PER_STREAM                           19.210
TOTAL_GAMES_STREAMED                             18.043
FOLLOWERS_GAINED_PER_STREAM                      13.033
LANGUAGEPortuguese                               12.086
TOTAL_VIEWS                                      11.604
RANK                                              9.363
X2ND_MOST_STREAMED_GAMEFINAL FANTASY XIV ONLINE   5.225
MOST_STREAMED_GAMEDota 2                          5.099
TYPEpersonality                                   3.182
MOST_STREAMED_GAMEAnimals Aquariums and Zoos      2.597
LANGUAGEEnglish                                   2.497
X2ND_MOST_STREAMED_GAMEAlbion Online              2.467
```

```
MOST_STREAMED_GAMESlots                          2.113
MOST_STREAMED_GAMEVariety                        2.054
MOST_STREAMED_GAMECasino                         1.838
```

```r
rfmodel21 = train(
  #choosing all the variables as predictors and TYPE(stream type)
  #as the variable that will be predicted.
  AVERAGE_STREAM_DURATION ~.,
  data = training,
  method = "rf",
  ntree=50
)
```

```r
pred = predict(rfmodel21, newdata = testing)
postResample(pred, testing$AVERAGE_STREAM_DURATION)
```

```
     RMSE  Rsquared       MAE
2.0533284 0.5990808 1.2889418
```

```r
varImp(rfmodel21)
```

```
rf variable importance

  only 20 most important variables shown (out of 322)

                                              Overall
TOTAL_TIME_STREAMED                           100.000
MOST_STREAMED_GAMEVirtual Casino               55.059
ACTIVE_DAYS_PER_WEEK                           50.681
TOTAL_FOLLOWERS                                30.916
TOTAL_GAMES_STREAMED                           18.372
AVG_VIEWERS_PER_STREAM                         17.353
AVG_GAMES_PER_STREAM                           17.075
LANGUAGEPortuguese                             12.691
FOLLOWERS_GAINED_PER_STREAM                    11.127
TOTAL_VIEWS                                    11.019
RANK                                            9.586
X2ND_MOST_STREAMED_GAMEFINAL FANTASY XIV ONLINE  6.065
MOST_STREAMED_GAMEDota 2                        5.366
LANGUAGEEnglish                                 4.727
MOST_STREAMED_GAMEAnimals Aquariums and Zoos    4.665
```

```
TYPEpersonality                                  3.848
MOST_STREAMED_GAMEVariety                        3.292
MOST_ACTIVE_DAYThursday                          2.325
MOST_STREAMED_GAMEGarena Free Fire               2.067
MOST_STREAMED_GAMECasino                         1.923
```

The best model was the one with **50 trees**. It has an RMSE close to 2.15, an R-squared value of about 0.55, and a mean absolute error of around 1.34. Since average stream duration is measured in hours, these values display a significant level of prediction error for a behavioral variable that is difficult to predict. The R-squared metric shows that the model explains a fair amount of the variability in stream duration.

For both models, different numbers of **trees** were tested. In both cases, for the classification and the regression tasks, models with a smaller number of trees (for example, **ntree = 50**) performed better than models with a larger number of trees (such as **ntree = 100** or **ntree = 300**). This is probably happening because of the high number of categorical levels in the dataset, such as the different games streamed (there are more than 100 distinct categories). When too many trees are used, noise is probably introduced, even though useful information is still contained in these variables, as we have seen using `varImp`. For this reason, the lighter models were retained. Some of these attempts are shown during the creation of the second model, which predicts the average streaming duration.

The output of the last code chunk shows that only a subset of predictors contributes to the prediction. This is reasonable given the large number of available predictors, with the output displaying the 20 most important variables out of a total of 322 (most of which come from factor variables). While there are many predictors, the model mainly relies on a smaller group of features, which supports the decision to keep the model simple and to use a small number of trees.

I obtained information about the caret package from:

The caret vignette:

https://cran.r-project.org/web/packages/caret/vignettes/caret.html

Caret Package cheat sheet:

https://rstudio.github.io/cheatsheets/caret.pdf